

친절한 SQL 튜닝 전범위 25문항

제작 기준

- 원천: **친절한 SQL 튜닝.pdf** 560페이지 전체 이미지 추출 및 한국어 OCR
- 범위: 1장 SQL 처리 과정과 I/O부터 7장 SQL 옵티마이저, 부록 SQL 분석 도구까지
- 출제 방식: 단순 개념 암기보다 **쿼리, 실행계획, I/O 통계, 인덱스 구성, 조인 메커니즘**을 해석하는 문제 중심
- 저작권 처리: OCR 원문 문장을 그대로 옮기지 않고 학습용 사례와 문항으로 재구성

커버리지

범위	핵심 내용	문항
1장 SQL 처리 과정과 I/O	파싱/최적화, 바인드 변수, 논리/물리 I/O, Full SCAN과 INDEX SCAN	1~3
2장 인덱스 기본	B-tree 탐색, RANGE SCAN 조건, 인덱스 확장 스캔	4~6
3장 인덱스 튜닝	테이블 랜덤 액세스, 부분범위 처리, 스캔 효율화, 옵션 조건, 인덱스 설계	7~12
4장 조인 튜닝	NL, Sort MERGE, HASH, 서브쿼리/뷰/스칼라 서브쿼리	13~16
5장 소트 튜닝	소트 수행, UNION/EXISTS, 인덱스 정렬 생략, Top N, Sort Area	17~19
6장 DML 튜닝	Array Processing, Direct Path I/O, 파티션 DML, Lock/채번	20~23
7장/부록	통계정보, 옵티마이저, AutoTrace, SQL Trace, DBMS_XPLAN, V\$SQL	24~25

문제 1. 하드 파싱과 바인드 변수

다음 애플리케이션 코드가 이벤트 시간대에 초당 수천 번 실행된다.

```
String sql =  
    "SELECT 고객ID, 고객명, 등급코드 " +  
    "FROM 고객 " +  
    "WHERE 로그인ID = '" + loginId + "'";  
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery(sql);
```

가장 적절한 튜닝 방향은?

- ① SQL 문자열을 매번 새로 만들지 말고 바인드 변수를 사용해 커서 공유 가능성을 높인다.
- ② 로그인ID 컬럼에 인덱스를 추가하면 하드 파싱 문제도 함께 사라진다.
- ③ **select-list** 컬럼 수를 줄이면 SQL 텍스트가 같아져 라이브러리 캐시 경합이 사라진다.
- ④ 동일 SQL이므로 대소문자와 공백 차이는 커서 공유에 영향을 주지 않는다.

문제 2. SQL Trace의 I/O 통계 해석

어떤 SQL의 Trace 통계가 다음과 같다.

Call	Count	Disk	Query	Current	Rows
Parse	1	0	0	0	0
Execute	1	0	5	10	1
Fetch	101	12000	480000	200	1000

이 SQL의 튜닝 방향을 판단한 설명으로 가장 적절한 것은?

- ① 물리 I/O는 12,000이고 논리 I/O는 480,215이므로, 우선 읽는 총 블록 수를 줄이는 방향을 검토한다.
- ② Disk보다 Query가 훨씬 크므로 캐시 효율은 양호하며, 인덱스나 조인 구조보다 메모리 증설을 먼저 검토한다.
- ③ Rows가 1,000건이면 Fetch Call이 101회인 점이 핵심이므로, 논리 I/O가 480,215여도 정상 범위로 본다.
- ④ Parse Count가 1이면 커서 공유는 성공한 상태이므로, 실행계획과 I/O 효율 문제는 별도로 분석하지 않는다.

문제 3. Full SCAN과 INDEX RANGE SCAN 선택

주문 테이블은 1억 건이고, 하루 평균 300만 건의 주문이 발생한다. 아래 SQL은 일 배치에서 하루치 주문을 모두 집계한다.

```
SELECT 주문일자, SUM(주문금액)
FROM 주문
WHERE 주문일자 = :target_date
GROUP BY 주문일자;
```

주문일자 인덱스가 있을 때 가장 적절한 판단은?

- ① 인덱스가 있으므로 INDEX RANGE SCAN을 우선 고정하고, 필요하면 힌트로 Full SCAN과 HASH 집계를 함께 방지한다.
- ② Full SCAN이 보이면 인덱스 통계가 누락된 신호이므로, 통계 수집 후에도 INDEX SCAN을 유도하고 ROWID 반복 비용은 무시한다.
- ③ 처리 대상이 300만 건 수준이면 ROWID 기반 테이블 랜덤 액세스가 커질 수 있어 Full SCAN 또는 파티션 Pruning을 검토한다.
- ④ GROUP BY가 있으면 정렬 또는 해시 집계가 필요하므로, 인덱스와 파티션 전략은 성능 판단에서 제외한다.

문제 4. B-tree 인덱스와 RANGE SCAN 가능 조건

다음 인덱스가 있다.

```
CREATE INDEX 고객_X1 ON 고객(생년월일);
```

아래 조건 중 일반적인 B-tree 인덱스 RANGE SCAN의 시작/종료 지점을 가장 명확히 찾을 수 있는 것은?

- ① WHERE SUBSTR(생년월일, 5, 2) = '05'처럼 월만 추출하면 찾을 값이 명확하므로 시작 지점도 명확하다.
- ② WHERE 생년월일 BETWEEN '20070101' AND '20070131'
- ③ WHERE 생년월일 LIKE '%0505%'처럼 포함 검색을 쓰면 원하는 값만 좁게 모여 Range Scan된다.
- ④ WHERE NVL(생년월일, '99991231') >= '20070101'처럼 NULL을 치환하면 원본 인덱스 순서가 그대로 유지된다.

문제 5. OR, IN 조건과 실행계획

다음 SQL과 실행계획을 보자. **고객_전화번호_IDX(전화번호)** 인덱스가 있다.

```
SELECT *  
FROM 고객  
WHERE 전화번호 IN (:tel1, :tel2, :tel3);
```

Id	Operation	Name
0	SELECT STATEMENT	
1	INLIST ITERATOR	
2	TABLE ACCESS BY INDEX ROWID	고객
3	INDEX RANGE SCAN	고객_전화번호_IDX

가장 적절한 설명은?

- ① IN 조건은 OR 조건의 축약 표현이므로, 여러 값이 있으면 옵티마이저는 항상 Full SCAN을 선택한다.
- ② 전화번호 인덱스가 유니크가 아니면 중복 후보가 생기므로, IN 조건에는 인덱스를 사용할 수 없다.
- ③ INLIST ITERATOR는 인덱스 탐색을 포기하고, IN-list 값을 메모리에서만 비교한다는 의미다.
- ④ IN-list 값마다 인덱스 RANGE SCAN을 반복하는 방식으로 처리될 수 있다.

문제 6. 인덱스 확장 스캔 방식

다음 SQL은 인덱스에 포함된 컬럼만 조회한다.

```
CREATE INDEX 주문_X1 ON 주문(고객ID, 주문일자, 주문금액);  
  
SELECT /*+ INDEX_FFS(o 주문_X1) */  
      고객ID, 주문일자, 주문금액  
FROM   주문 o  
WHERE  주문일자 BETWEEN :d1 AND :d2;
```

INDEX FAST FULL SCAN이 선택되었을 때 옳은 설명은?

- ① 인덱스의 논리적 리프 연결 순서대로 읽으므로 **고객ID, 주문일자** 정렬 결과가 보장된다.
- ② 인덱스 세그먼트를 테이블처럼 읽을 수 있지만, 인덱스 키 순서 정렬은 보장하지 않는다.
- ③ 반드시 테이블을 ROWID로 방문해야 하므로 인덱스만 읽고 처리할 수 없다.
- ④ Single Block I/O만 사용하므로 대량 스캔에 불리하다.

문제 7. 테이블 랜덤 액세스와 클러스터링 팩터

다음 두 인덱스 중 하나를 선택해야 한다.

주문_X1(고객ID)
- 예상 인덱스 스캔 건수: 10,000
- 클러스터링 팩터: 매우 나쁨

주문_X2(주문일자)
- 예상 인덱스 스캔 건수: 40,000
- 클러스터링 팩터: 매우 좋음

조회 SQL은 테이블 컬럼을 여러 개 읽어야 해서 **TABLE ACCESS BY INDEX ROWID**가 필요하다. 가장 적절한 판단은?

- ① 스캔 건수가 적은 **주문_X1**이 항상 빠르므로, 클러스터링 팩터보다 선택도만 우선 비교한다.
- ② 클러스터링 팩터는 Full SCAN에서 멀티블록 I/O 효율을 판단하는 지표이므로 인덱스 선택과 테이블 랜덤 액세스와는 무관하다.
- ③ 클러스터링 팩터가 나쁜 인덱스는 같은 테이블 블록을 반복 방문할 수 있으므로 테이블 액세스 비용까지 비교해야 한다.
- ④ 인덱스를 타면 ROWID가 물리 포인터처럼 바로 레코드를 가리키므로 테이블 액세스 비용은 거의 없다고 본다.

문제 8. 부분범위 처리와 화면 응답 속도

상담 화면에서 특정 고객의 최근 거래 30건만 먼저 보여주려고 한다.

```
SELECT *  
FROM (  
  SELECT 거래번호, 고객ID, 거래일시, 거래금액  
  FROM 거래  
  WHERE 고객ID = :cust_id  
  ORDER BY 거래일시 DESC  
)  
WHERE ROWNUM <= 30;
```

가장 적절한 인덱스는?

- ① 거래_X1(거래금액, 거래일시)
- ② 거래_X2(거래일시, 고객ID)
- ③ 거래_X3(고객ID, 거래일시 DESC)
- ④ 거래_X4(고객ID, 거래금액, 거래일시)

문제 9. 액세스 조건과 필터 조건

다음 인덱스가 있다.

```
CREATE INDEX 가입상품_X1 ON 가입상품(회사코드, 지역코드, 상품명);
```

아래 SQL에서 **지역코드** 값을 항상 입력한다고 가정한다.

```
SELECT 고객ID, 상품명  
FROM 가입상품  
WHERE 회사코드 = :company  
AND 지역코드 LIKE :region || '%'  
AND 상품명 LIKE :product || '%';
```

지역코드 = :region으로 바꾸었을 때 기대할 수 있는 가장 중요한 효과는?

- ① **지역코드**가 테이블 필터 조건으로만 바뀐다.
- ② LIKE와 등치 조건은 B-tree 인덱스에서 항상 같은 스캔 범위를 만든다.
- ③ 인덱스 선두 컬럼인 **회사코드**를 더 이상 사용할 수 없다.
- ④ 뒤 컬럼인 **상품명**까지 인덱스 액세스 범위 축소에 더 잘 활용될 수 있다.

문제 10. BETWEEN을 IN-list로 바꾸는 경우

인덱스가 다음과 같다.

```
CREATE INDEX 일별거래_X1 ON 일별거래(거래일자, 시장구분, 종목코드);
```

아래 SQL은 특정 3일의 특정 시장, 특정 종목을 조회한다.

```
SELECT *  
FROM 일별거래  
WHERE 거래일자 BETWEEN :d1 AND :d3  
AND 시장구분 = :market  
AND 종목코드 = :item;
```

:d1, :d2, :d3가 정확히 3개 날짜일 때 다음처럼 바꾸는 튜닝이 효과적일 수 있는 이유는?

```
WHERE 거래일자 IN (:d1, :d2, :d3)  
AND 시장구분 = :market  
AND 종목코드 = :item
```

- ① 거래일자 조건이 사라지는 효과가 있어 옵티마이저가 테이블 Full SCAN을 더 쉽게 선택하기 때문이다.
- ② 각 날짜별로 **시장구분, 종목코드**까지 더 좁은 인덱스 탐색 범위를 만들 수 있기 때문이다.
- ③ IN-list는 선두 컬럼을 여러 값으로 나누므로 후행 컬럼을 모두 필터 조건으로 강등시키기 때문이다.
- ④ BETWEEN은 하나의 연속 구간만 읽으므로 값 개수와 후행 조건에 관계없이 항상 더 적은 블록을 읽는다.

문제 11. 옵션 조건 처리 방식

검색 화면에서 **고객ID**는 선택 입력이고 **거래일자**는 필수 입력이다. 다음 SQL을 하나로 통합하려고 한다.

```
SELECT *  
FROM 거래  
WHERE (:cust_id IS NULL OR 고객ID = :cust_id)  
AND 거래일자 BETWEEN :d1 AND :d2;
```

인덱스 후보는 **거래_X1(고객ID, 거래일자)**와 **거래_X2(거래일자, 고객ID)**이다. 가장 적절한 설명은?

- ① 고객ID가 선택 입력이면 SQL을 분기하거나 UNION ALL로 나누어 각 경우에 맞는 액세스 경로를 만드는 방식을 검토한다.
- ② OR 옵션 조건은 **고객ID**가 입력된 경우에만 평가되므로, **거래_X1**의 선두 컬럼 액세스 조건으로 안정적으로 작동한다.
- ③ **거래_X1**만 있으면 고객ID를 입력하지 않은 경우에도 거래일자 조건이 후행 컬럼에서 충분히 좁은 범위를 만들고 RANGE SCAN이 안정적이다.
- ④ 옵션 조건은 실행 시점에 값이 결정되므로 인덱스 컬럼 순서보다 바인드 변수 타입을 우선 조정해야 한다.

문제 12. 인덱스 설계와 중복 제거

다음 인덱스들이 있다.

```
고객_X1(고객ID)  
고객_X2(고객ID, 가입일자)  
고객_X3(고객ID, 가입일자, 상태코드)  
고객_X4(가입일자, 상태코드)
```

주요 SQL은 **고객ID = :id** 단건 조회, **고객ID = :id AND 가입일자 BETWEEN ...** 조회, **가입일자 BETWEEN ... AND 상태코드 = ...** 집계다. 가장 적절한 판단은?

- ① **고객_X1**, **고객_X2**, **고객_X3**는 선두 컬럼이 같아도 길이가 다르므로 옵티마이저가 항상 서로 다른 용도로 쓰여 중복으로 보지 않는다.
- ② **고객_X4**는 **고객ID**가 없으므로 고객 테이블의 주요 SQL과 무관하며, 가입일자 집계에도 도움이 되지 않는 중복 인덱스다.
- ③ 인덱스가 많을수록 조회 후보가 늘어나므로 DML 유지 비용보다 옵티마이저 선택 폭 확대를 우선한다.
- ④ **고객_X3**가 **고객_X1**, **고객_X2** 역할을 상당 부분 대체할 수 있는지 검토하고, DML 부하까지 고려해 중복 제거를 판단한다.

문제 13. NL 조인 실행계획 해석

다음 실행계획을 보자.

Id	Operation	Name	Starts	A-Rows	Buffers
0	SELECT STATEMENT		1	500	82000
1	NESTED LOOPS		1	500	82000
2	TABLE ACCESS BY INDEX ROWID	주문	1	10000	12000
3	INDEX RANGE SCAN	주문_X1	1	10000	2000
4	TABLE ACCESS BY INDEX ROWID	주문상세	10000	500	70000
5	INDEX RANGE SCAN	주문상세_X1	10000	500	40000

가장 적절한 분석은?

- ① 주문상세_X1의 Starts가 10,000이므로 선행 주문 결과마다 후행 인덱스 탐색이 반복되었다.
- ② 주문상세의 A-Rows가 500이므로 최종 반환 건수는 작고, 후행 인덱스 반복 비용도 작다고 판단한다.
- ③ NL 조인은 인덱스 기반 조인이므로 Buffers보다 A-Rows를 우선 보며, 버퍼 읽기는 대체로 무시할 수 있다.
- ④ 주문의 A-Rows가 10,000이어도 후행 인덱스가 있으면 랜덤 액세스가 순차 I/O로 바뀌어 병목이 사라진다.

문제 14. Sort MERGE JOIN 선택 상황

두 테이블을 조인하는 SQL에서 양쪽 입력 집합이 이미 조인 컬럼 순서로 정렬되어 있거나, 각각 인덱스 Full SCAN으로 정렬 순서를 얻을 수 있다. 조인 결과는 대량이며 후행 랜덤 액세스를 반복하는 NL 조인은 부담스럽다.

가장 적절한 판단은?

- ① Sort MERGE JOIN은 이름에 Sort가 포함되어 있으므로 인덱스 정렬 순서가 있어도 항상 새 정렬을 수행한다.
- ② Sort MERGE JOIN은 조인 전에 양쪽을 정렬하지만, 정렬은 디스크가 담당하므로 PGA 사용량과 무관하다.
- ③ 정렬 부담이 작거나 이미 정렬된 입력을 활용할 수 있다면 Sort MERGE JOIN이 합리적일 수 있다.
- ④ Sort MERGE JOIN은 조인 조건이 약하거나 결과가 대량일수록 항상 HASH JOIN보다 안정적인 선택이다.

문제 15. HASH JOIN과 Build Input

다음 SQL은 대량 집합을 조인한다.

```
SELECT /*+ USE_HASH(o) */
      c.고객등급, SUM(o.주문금액)
FROM   고객 c
      JOIN 주문 o ON o.고객ID = c.고객ID
WHERE  o.주문일자 BETWEEN :d1 AND :d2
GROUP BY c.고객등급;
```

고객은 50만 건, 기간 조건을 만족하는 주문은 2천만 건이다. HASH JOIN에 대한 설명으로 가장 적절한 것은?

- ① Build Input은 SQL에 나중에 적은 테이블이 되므로, FROM 절 순서만 조정해도 메모리 사용량을 예측할 수 있다.
- ② 일반적으로 더 작은 입력인 **고객**을 Build Input으로 삼아 해시 테이블을 만드는 것이 유리할 수 있다.
- ③ HASH JOIN은 인덱스 탐색을 반복하지 않기 때문에 대량 처리에서는 항상 NL 조인보다 빠르다고 단정할 수 있다.
- ④ HASH JOIN은 해시 테이블을 만들지만 Temp로 넘기지 않으므로, 메모리 크기보다 조인 컬럼 NDV가 중요하다.

문제 16. 서브쿼리 변환과 스칼라 서브쿼리

다음 SQL을 보자.

```
SELECT c.고객ID, c.고객명,
      (SELECT MAX(o.주문일자)
       FROM 주문 o
       WHERE o.고객ID = c.고객ID) AS 최근주문일자
FROM   고객 c
WHERE  c.상태코드 = 'A';
```

가장 적절한 설명은?

- ① 스칼라 서브쿼리는 바깥쪽 고객 건수만큼 실행되므로, 주문 테이블 Full SCAN이 반복된다고 보는 것이 안전하다.
- ② 옵티마이저는 상황에 따라 서브쿼리를 조인 형태로 변환하거나, 인덱스와 캐싱 효과를 활용해 처리할 수 있다.
- ③ 서브쿼리는 SELECT 절에 있으면 실행계획에서 조인으로 변환되지 않고 항상 별도 필터 단계로만 남는다.
- ④ 스칼라 서브쿼리는 여러 건을 반환해도 가장 먼저 찾은 한 건만 쓰므로, MAX 같은 집계 함수는 성능상 불필요하다.

문제 17. UNION, EXISTS, 소트 발생

다음 두 SQL은 같은 업무 요건을 구현하려고 작성한 후보안이다.

```
-- A안
SELECT 고객ID FROM 온라인고객
UNION
SELECT 고객ID FROM 오프라인고객;

-- B안
SELECT 고객ID FROM 온라인고객
UNION ALL
SELECT 고객ID FROM 오프라인고객;
```

두 집합에 중복 고객ID가 없다는 업무 규칙이 보장될 때 가장 적절한 판단은?

- ① 중복이 없더라도 **UNION ALL**은 데이터 품질 검증을 생략하므로, 업무 규칙이 명확해도 사용하지 않는 편이 안전하다.
- ② B안은 두 집합을 단순 연결하므로 중복 제거 비용은 없지만, 중복이 없을 때도 결과 건수는 A안보다 적어진다.
- ③ **UNION**과 **UNION ALL**은 모두 집합 연산이므로 옵티마이저가 중복 여부를 판단해 같은 실행 방식으로 바꾼다.
- ④ A안은 중복 제거가 필요하므로 불필요한 소트 또는 해시 중복 제거가 발생할 수 있다.

문제 18. 인덱스를 이용한 ORDER BY, MIN/MAX 생략

다음 인덱스가 있다.

```
CREATE INDEX 체결_X1 ON 체결(종목코드, 체결일시 DESC, 체결가격);
```

아래 SQL의 기대 실행 방식으로 가장 적절한 것은?

```
SELECT *
FROM (
  SELECT 체결번호, 종목코드, 체결일시, 체결가격
  FROM 체결
  WHERE 종목코드 = :code
  ORDER BY 체결일시 DESC
)
WHERE ROWNUM <= 5;
```

- ① 인덱스가 DESC 컬럼을 포함하면 정렬 전용으로만 쓸 수 있고, 조건 검색을 위한 RANGE SCAN은 불가능하다.
- ② **ORDER BY**가 있으면 인덱스 순서와 무관하게 Sort Order By가 필요하므로 전체 후보를 정렬해야 한다.
- ③ **종목코드** 등치 조건으로 인덱스 범위를 잡고 **체결일시 DESC** 순서대로 읽다가 5건에서 멈출 수 있다.
- ④ **ROWNUM <= 5**는 정렬이 끝난 뒤 적용되는 필터이므로, Stopkey 여부와 인덱스 순서는 성능에 영향을 주지 않는다.

문제 19. Sort Area를 적게 쓰는 SQL 작성

아래 두 SQL 중 Sort Area 사용량을 줄이는 방향으로 더 적절한 것은?

```
-- A안
SELECT LPAD(상품번호, 30) || LPAD(상품명, 100) || TO_CHAR(주문일시, 'yyyymmdd hh24:mi:ss') AS line
FROM 주문상품
WHERE 주문일시 BETWEEN :d1 AND :d2
ORDER BY 상품번호;

-- B안
SELECT LPAD(상품번호, 30) || LPAD(상품명, 100) || TO_CHAR(주문일시, 'yyyymmdd hh24:mi:ss') AS line
FROM (
    SELECT 상품번호, 상품명, 주문일시
    FROM 주문상품
    WHERE 주문일시 BETWEEN :d1 AND :d2
    ORDER BY 상품번호
);
```

Sort Area 사용량을 줄이는 개선안으로 가장 적절한 것은?

- ① B안이 더 적절하다. 정렬은 원본 컬럼 기준으로 수행하고 포매팅은 정렬 이후로 미룰 수 있다.
- ② A안이 더 적절하다. 정렬 전에 문자열을 완성하면 정렬 후 추가 연산이 줄어 전체 메모리 사용량도 감소한다.
- ③ 두 SQL은 같은 행과 같은 컬럼을 최종 출력하므로, 옵티마이저 입장에서는 정렬 대상 데이터 폭도 항상 동일하다.
- ④ Sort Area는 SQL 작성 방식보다 PGA 설정값으로 결정되므로, 표현식 위치를 바꿔도 Temp 사용량은 같다.

문제 20. Array Processing과 Database Call

다음 프로그램은 10만 건을 한 건씩 INSERT한다.

```
for (Row r : rows) {
    PreparedStatement ps = conn.prepareStatement(
        "INSERT INTO 주문_임시(주문번호, 고객ID, 주문금액) VALUES (?, ?, ?)");
    ps.setLong(1, r.orderNo);
    ps.setString(2, r.customerId);
    ps.setBigDecimal(3, r.amount);
    ps.executeUpdate();
}
```

가장 적절한 개선 방향은?

- ① 한 건씩 처리하면 각 로우의 Lock 유지 시간이 짧아지고 실패 지점도 명확해지므로, 대량 INSERT에서도 전체 처리량이 가장 안정적이다.
- ② PreparedStatement를 재사용하고 배치 바인딩/Array Processing으로 Database Call 횟수를 줄인다.
- ③ 인덱스를 충분히 추가하면 INSERT 대상 블록과 중복 여부를 더 빨리 찾을 수 있으므로 Database Call 횟수도 함께 감소한다.
- ④ Commit을 마지막에 한 번만 수행하면 로그 기록과 Lock 관리가 사라지므로, 장애 복구 측면에서도 가장 안전하다.

문제 21. Direct Path Insert

대량 이관 테이블에 야간 배치로 5천만 건을 적재한다. 기존 데이터와 동시 조회/갱신 충돌은 없고, 적재 후 인덱스 생성과 통계 수집을 수행할 수 있다.

가장 적절한 검토 사항은?

- ① Direct Path Insert를 사용하면 통계정보 수집이 필요 없다.
- ② Direct Path Insert는 항상 OLTP 단건 INSERT에도 적합하다.
- ③ Direct Path Insert는 Redo, Undo, Lock, 인덱스 유지 비용을 전혀 발생시키지 않는다.
- ④ **APPEND** 또는 병렬 Direct Path Insert를 검토해 버퍼 캐시 경유 부담을 줄일 수 있다.

문제 22. 파티션과 대량 DML

주문 테이블은 주문일자 기준 RANGE 파티션이고, 과거 1개월 데이터를 삭제해야 한다. 다음 중 가장 적절한 전략은?

- ① 로컬 인덱스는 파티션과 1:1로 대응하므로, 파티션 구조를 변경하면 글로벌 인덱스보다 항상 더 쉽게 Unusable이 된다.
- ② 파티션 테이블이라도 인덱스 정합성을 지키려면 과거 데이터 삭제는 Row 단위 DELETE로 처리해야 한다.
- ③ 월 파티션이 분리되어 있으면 **DELETE**보다 파티션 **TRUNCATE** 또는 **DROP** 방식이 훨씬 빠를 수 있다.
- ④ 파티션 Pruning은 조회 대상 세그먼트를 줄이는 기능이므로 SELECT에는 유효하지만 DELETE/INSERT에는 적용되지 않는다.

문제 23. Lock, 트랜잭션, 채번 방식

다음 방식으로 주문번호를 채번한다.

```
SELECT MAX(주문번호) + 1  
FROM 주문  
WHERE 주문일자 = :today;
```

동시 주문이 매우 많은 시스템에서 이 방식의 문제로 가장 적절한 것은?

- ① 같은 번호를 계산한 트랜잭션들이 충돌할 수 있어 직렬화나 Lock 경합 문제가 생길 수 있다.
- ② **max + 1** 방식은 날짜별 마지막 번호를 직접 확인하므로, 캐시된 시퀀스보다 업무 규칙과 확장성 측면에서 유리하다.
- ③ 주문일자와 주문번호 인덱스가 있으면 최댓값 탐색이 빨라지고, 같은 번호를 계산하는 동시성 문제도 함께 제거된다.
- ④ 채번은 INSERT 전에 수행되는 조회일 뿐이므로, 실제 INSERT 처리량과 Lock 경합에는 직접 영향을 주지 않는다.

문제 24. 선택도, 카디널리티, 통계정보

상품 테이블은 1,000만 건이고 상품분류코드의 NDV는 100이다. 히스토그램이 없고 값 분포가 균일하다고 가정하면 다음 조건의 단순 카디널리티 추정값으로 가장 가까운 것은?

```
WHERE 상품분류코드 = :cls
```

- ① 약 1만 건
- ② 약 1,000만 건
- ③ 약 100만 건
- ④ 약 10만 건

문제 25. SQL 분석 도구 선택

운영 중 특정 SQL이 간헐적으로 느리다. 예상 실행계획만으로는 원인을 알 수 없고, 실제 실행 시 각 오퍼레이션의 **A-Rows**, **Buffers**, **Starts**를 보고 싶다.

가장 적절한 방법은?

- ① **EXPLAIN PLAN**에 표시되는 예상 Rows와 Cost를 확인하면, 실제 실행 시 Row Source 통계도 같은 값으로 해석할 수 있다.
- ② **dbms_xplan.display_cursor**에 **ALLSTATS LAST** 형식으로 실제 커서의 Row Source 통계를 확인한다.
- ③ **V\$SQL**의 누적 **buffer_gets**, **executions**를 나누어 평균 I/O를 구하면 실행계획 단계별 병목까지 식별할 수 있다.
- ④ AutoTrace의 예상 실행계획은 SQL을 실행하지 않고도 통계를 보여주므로, 간헐적 지연 분석에 가장 적합하다.

정답 및 해설

정답 한 번에 보기

문제	정답	문제	정답	문제	정답	문제	정답	문제	정답
1	①	2	①	3	③	4	②	5	④
6	②	7	③	8	③	9	④	10	②
11	①	12	④	13	①	14	③	15	②
16	②	17	④	18	③	19	①	20	②
21	④	22	③	23	①	24	④	25	②

문제 1 해설

정답: ①

핵심 개념

SQL 최적화는 CPU를 많이 쓰는 무거운 작업이며, 하드 파싱 결과는 라이브러리 캐시에 커서로 저장된다. SQL 텍스트가 매번 달라지면 의미가 같아도 커서 공유가 어려워진다.

정답 도출

문자열 결합 방식은 로그인ID 값마다 서로 다른 SQL 텍스트를 만든다. 바인드 변수로 바꾸면 SQL 텍스트가 안정되어 하드 파싱과 라이브러리 캐시 경합을 줄일 수 있다.

```
SELECT 고객ID, 고객명, 등급코드
FROM 고객
WHERE 로그인ID = :login_id;
```

오답 분석

선택지	이유
②	인덱스는 데이터 액세스 문제를 줄일 수 있지만, SQL 텍스트가 달라지는 하드 파싱 문제를 직접 해결하지 못한다.
③	select-list 축소는 전송량과 테이블 액세스에는 영향을 줄 수 있지만 커서 공유의 핵심은 SQL 텍스트와 바인드 사용이다.
④	DBMS는 SQL 텍스트를 기준으로 커서를 찾으므로 대소문자, 공백, 주석, 스키마 표기 차이가 별도 커서를 만들 수 있다.

정리

인덱스 튜닝 전에 SQL 공유와 재사용 구조를 먼저 안정화해야 한다. OLTP 로그인, 조회, 주문처럼 반복 호출되는 SQL은 바인드 변수가 기본이다.

문제 2 해설

정답: ①

핵심 개념

SQL Trace의 **Query + Current**는 논리 I/O, **Disk**는 물리 I/O로 해석한다. SQL 튜닝의 핵심은 통제 가능한 논리 I/O를 줄이는 것이다.

정답 도출

이 SQL의 전체 논리 I/O는 다음과 같다.

Query (5 + 480,000) + Current (10 + 200) = 480,215

물리 I/O는 12,000이다. 물리 I/O가 상대적으로 작아 보여도, 1,000건을 가져오기 위해 48만 블록 이상을 읽는다면 인덱스/조인/소트/필터 조건을 재검토해야 한다.

오답 분석

선택지	이유
②	버퍼 캐시 히트율이 높아도 같은 블록을 비효율적으로 반복 읽으면 SQL은 느릴 수 있다.
③	반환 건수만으로 정상 여부를 판단할 수 없다. 핵심은 결과를 얻기 위해 읽은 블록 수다.
④	Parse Count가 작다는 것은 파싱 횟수 문제와 관련될 뿐, 실행계획이나 I/O 효율을 보장하지 않는다.

정리

BCHR보다 개별 SQL의 논리 I/O가 더 직접적인 튜닝 지표다.

문제 3 해설

정답: ③

핵심 개념

인덱스는 소량 데이터를 빨리 찾는 도구다. 대량 데이터를 인덱스로 찾은 뒤 테이블을 반복 방문하면 Full SCAN보다 느릴 수 있다.

정답 도출

하루치 데이터가 300만 건이면 **주문일자** 인덱스로 300만 ROWID를 얻고 테이블을 랜덤 액세스할 수 있다. 이 경우 대량 순차 처리 관점에서 Full SCAN, 파티션 Pruning, 병렬 처리, 집계용 인덱스 등을 함께 검토한다.

오답 분석

선택지	이유
①	인덱스 존재가 최적 경로를 보장하지 않는다.
②	Full SCAN은 대량 처리에서 정상적이고 유리한 선택일 수 있다.
④	GROUP BY가 있어도 인덱스나 파티션, Full SCAN 중 비용이 낮은 경로를 선택할 수 있다.

정리

Full SCAN이 나쁜 것이 아니라, 처리량과 액세스 패턴에 맞지 않는 경로가 나쁘다.

문제 4 해설

정답: ②

핵심 개념

B-tree RANGE SCAN은 정렬된 인덱스에서 시작 지점과 종료 지점을 찾을 수 있어야 한다. 컬럼을 함수로 가공하거나 중간 문자열을 찾으면 범위를 직접 잡기 어렵다.

정답 도출

생년월일 BETWEEN '20070101' AND '20070131'은 인덱스에 저장된 원본 값 기준으로 범위를 지정한다. 따라서 시작 위치와 종료 위치가 명확하다.

오답 분석

선택지	이유
①	substr 로 컬럼을 가공하면 일반 인덱스의 원본 정렬 구조를 직접 활용하기 어렵다.
③	앞쪽 와일드카드 검색은 값이 인덱스 전체에 흩어져 있어 시작 지점을 잡기 어렵다.
④	nvl 로 컬럼을 가공했으므로 함수 기반 인덱스가 없다면 정상적인 RANGE SCAN이 어렵다.

정리

인덱스 컬럼은 가능한 한 가공하지 말고 원본 컬럼 기준의 범위로 표현한다.

문제 5 해설

정답: ④

핵심 개념

IN 조건은 OR 조건의 다른 표현이지만, 옵티마이저가 IN-list Iterator로 각 값에 대해 RANGE SCAN을 반복하는 방식으로 처리할 수 있다.

정답 도출

실행계획에 **INLIST ITERATOR**와 **INDEX RANGE SCAN**이 함께 나타난다. 이는 **:tel1**, **:tel2**, **:tel3** 각각에 대해 전화번호 인덱스 범위를 탐색하는 구조로 이해할 수 있다.

오답 분석

선택지	이유
①	IN 조건도 적절한 인덱스가 있으면 RANGE SCAN 반복으로 처리될 수 있다.
②	유니크 인덱스가 아니어도 RANGE SCAN은 가능하다.
③	실행계획에 명확히 INDEX RANGE SCAN 이 있다.

정리

OR/IN 조건은 실행계획에서 쿼리 변환, Concatenation, INLIST ITERATOR 여부를 확인해야 한다.

문제 6 해설

정답: ②

핵심 개념

INDEX Fast Full SCAN은 인덱스 트리의 논리 순서를 따라가는 것이 아니라 인덱스 세그먼트를 빠르게 읽는다. 정렬 순서를 보장하지 않는 대신 대량 인덱스 스캔에 유리할 수 있다.

정답 도출

조희 컬럼이 모두 **주문_X1**에 포함되어 있으므로 테이블 방문 없이 인덱스만 읽을 수 있다. 그러나 Fast Full SCAN은 물리적 저장 순서에 가깝게 읽으므로 **ORDER BY** 생략 근거가 되지 않는다.

오답 분석

선택지	이유
①	정렬 순서를 보장하는 쪽은 보통 RANGE SCAN 또는 INDEX Full SCAN 계열이다.
③	필요한 컬럼이 모두 인덱스에 있으면 테이블 액세스를 생략할 수 있다.
④	Fast Full SCAN은 Multiblock I/O와 병렬 처리에 유리할 수 있다.

정리

INDEX FULL SCAN과 INDEX FAST FULL SCAN은 이름이 비슷하지만 정렬 보장과 I/O 방식이 다르다.

문제 7 해설

정답: ③

핵심 개념

인덱스 튜닝은 인덱스 스캔량뿐 아니라 테이블 랜덤 액세스 비용까지 봐야 한다. 클러스터링 팩터가 나쁘면 인덱스 순서와 테이블 저장 순서가 맞지 않아 블록 방문이 늘어난다.

정답 도출

주문_X1은 인덱스 스캔 건수는 적지만 클러스터링 팩터가 매우 나쁘다. 반면 주문_X2는 더 많이 스캔해도 테이블 블록 접근이 연속적일 수 있다. 따라서 실행계획의 Buffers, 테이블 액세스 횟수, 실제 응답 시간을 비교해야 한다.

오답 분석

선택지	이유
①	인덱스 스캔 건수만으로 전체 비용을 판단할 수 없다.
②	클러스터링 팩터는 인덱스 ROWID로 테이블을 방문할 때 특히 중요하다.
④	ROWID는 곧바로 메모리 포인터처럼 연결되는 주소가 아니므로 액세스 비용이 발생한다.

정리

실행계획에서 TABLE ACCESS BY INDEX ROWID를 보면 랜덤 액세스 비용을 반드시 떠올려야 한다.

문제 8 해설

정답: ③

핵심 개념

부분범위 처리는 전체 결과를 모두 만들지 않고 필요한 일부를 먼저 반환하는 전략이다. Top N 화면에서는 조건과 정렬을 동시에 만족하는 인덱스가 중요하다.

정답 도출

고객ID = :cust_id로 특정 고객 범위를 찾고, 그 범위 안에서 거래일시 DESC 순서대로 읽으면 최근 거래부터 반환할 수 있다. ROWNUM <= 30에서 멈추므로 초기 응답 속도가 빨라진다.

오답 분석

선택지	이유
①	고객 조건을 이용할 수 없고 정렬 목적과도 맞지 않는다.
②	거래일시가 선두이면 전체 거래 시간 순서에서 고객ID를 필터링해야 할 수 있다.
④	고객ID 이후 거래금액이 끼어 있어 거래일시 정렬 순서를 직접 활용하기 어렵다.

정리

Top N 인덱스는 **필수 등치 조건 + 정렬 컬럼** 순서를 우선 검토한다.

문제 9 해설

정답: ④

핵심 개념

결합 인덱스에서 앞 컬럼이 범위 조건이 되면 뒤 컬럼이 스캔 범위를 줄이는 힘이 약해질 수 있다. 등치 조건은 후속 컬럼의 군집성을 살리는 데 유리하다.

정답 도출

지역코드 LIKE :region || '%'는 입력값이 정확한 코드라도 범위 조건이다. 지역코드 = :region으로 바꾸면 (회사코드, 지역코드)까지 정확히 좁힌 뒤 상품명 LIKE 'xxx%' 범위를 더 잘 활용할 수 있다.

오답 분석

선택지	이유
①	등치 조건은 오히려 액세스 조건으로 쓰이기 쉽다.
②	LIKE는 패턴에 따라 범위가 넓어질 수 있고 등치 조건과 같지 않다.
③	선두 컬럼 회사코드 등치 조건은 그대로 사용할 수 있다.

정리

입력값이 필수이고 정확히 일치해야 한다면 습관적인 LIKE보다 = 조건이 인덱스 효율에 유리하다.

문제 10 해설

정답: ②

핵심 개념

선두 컬럼이 범위 조건이면 후행 컬럼의 탐색 효율이 제한될 수 있다. 범위가 소수의 이산 값이라면 IN-list로 분해해 후행 등치 조건을 더 잘 활용할 수 있다.

정답 도출

BETWEEN은 :d1부터 :d3까지 하나의 연속 구간을 스캔한다. 반면 IN-list는 세 날짜 각각에 대해 (거래일자, 시장구분, 종목코드)의 더 좁은 구간을 탐색할 수 있다.

오답 분석

선택지	이유
①	거래일자 조건은 사라지지 않는다.
③	IN-list는 오히려 반복 RANGE SCAN으로 후행 조건을 더 잘 살릴 수 있다.
④	BETWEEN과 IN-list의 우열은 값 개수, 분포, 후행 조건에 따라 달라진다.

정리

BETWEEN을 IN-list로 바꾸는 튜닝은 값 개수가 적을 때만 검토한다. 값이 많으면 반복 탐색 비용이 더 커질 수 있다.

문제 11 해설

정답: ①

핵심 개념

옵션 조건을 OR로 한 SQL에 몰아넣으면 인덱스 액세스 조건으로 쓰기 어려워질 수 있다. 조건 입력 여부에 따라 다른 최적 경로가 필요하다.

정답 도출

고객ID를 입력한 경우에는 (고객ID, 거래일자) 인덱스가 유리할 수 있고, 입력하지 않은 경우에는 (거래일자, 고객ID) 또는 날짜 파티션/Full SCAN이 유리할 수 있다. SQL 분기나 UNION ALL로 두 경로를 분리하면 각 경우에 맞는 실행계획을 만들기 쉽다.

오답 분석

선택지	이유
②	:cust_id IS NULL OR 고객ID = :cust_id는 일반적으로 고객ID 인덱스 액세스 조건으로 쓰기 어렵다.
③	고객ID가 NULL이면 선두 컬럼 조건이 없어져 거래_X1로 소량 스캔을 보장할 수 없다.
④	옵션 조건은 인덱스 컬럼 순서와 SQL 분기 전략에 큰 영향을 준다.

정리

검색 화면 SQL은 개발 편의성보다 입력 조합별 실행계획 안정성이 더 중요할 때가 많다.

문제 12 해설

정답: ④

핵심 개념

선두 컬럼이 같은 인덱스는 상위 인덱스가 하위 인덱스 역할을 대체할 수 있는지 검토해야 한다. 중복 인덱스는 DML 부하와 저장 공간을 늘린다.

정답 도출

고객_X3(고객ID, 가입일자, 상태코드)는 고객ID 단건 조회와 고객ID + 가입일자 조회에도 사용 가능하다. 다만 인덱스 크기, 클러스터링 팩터, Unique 여부, 자주 쓰는 SQL의 응답 요구를 확인한 뒤 제거 여부를 결정한다. 고객_X4는 가입일자/상태코드 집계용으로 별도 가치가 있다.

오답 분석

선택지	이유
①	같은 선두 컬럼 인덱스가 모두 필요한 것은 아니다.
②	고객_X4는 가입일자와 상태코드를 조건으로 하는 집계 SQL에 유용할 수 있다.
③	인덱스가 많으면 DML마다 유지 비용이 증가한다.

정리

인덱스 설계는 추가보다 정리도 중요하다. SQL별 필요성과 DML 비용을 함께 본다.

문제 13 해설

정답: ①

핵심 개념

NL 조인은 선행 집합의 각로우마다 후행 집합을 반복 탐색한다. Starts는 특정 오퍼레이션이 몇 번 실행되었는지 보여준다.

정답 도출

주문에서 10,000건이 나오고, 주문상세_X1의 Starts도 10,000이다. 이는 선행 주문 10,000건 각각에 대해 후행 주문상세 인덱스를 탐색했다는 뜻이다. A-Rows가 최종 500이어도 반복 탐색 과정에서 Buffers가 70,000까지 늘었다.

오답 분석

선택지	이유
②	결과 건수만 적어도 반복 탐색과 버퍼 읽기가 많으면 비효율이다.
③	NL 조인은 반복 랜덤 액세스로 Buffers가 커질 수 있다.
④	후행 인덱스가 있어도 반복 횟수가 많으면 병목이 된다.

정리

NL 조인 튜닝은 **선행 건수 × 후행 접근 비용**을 줄이는 게임이다.

문제 14 해설

정답: ③

핵심 개념

Sort MERGE JOIN은 양쪽 입력을 조인 키 순서로 정렬해 병합한다. 이미 정렬된 경로를 활용하거나 대량 조인에서 랜덤 액세스를 피할 때 고려할 수 있다.

정답 도출

문제 조건은 이미 정렬된 입력을 얻기 쉬우며, NL의 반복 랜덤 액세스가 부담스럽다. 이런 상황에서는 Sort MERGE JOIN이 선택될 수 있다. 단, 정렬 비용과 PGA/Temp 사용 가능성을 함께 봐야 한다.

오답 분석

선택지	이유
①	입력이 이미 정렬되어 있으면 정렬 부담이 작거나 생략될 수 있다.
②	Sort MERGE JOIN은 정렬 과정에서 PGA와 Temp를 사용할 수 있다.
④	조인 조건과 데이터량에 따라 적합성이 달라진다.

정리

조인 방식은 고정 답이 없다. 데이터량, 정렬 여부, 랜덤 액세스 비용을 함께 판단한다.

문제 15 해설

정답: ②

핵심 개념

HASH JOIN은 한쪽 입력으로 해시 테이블을 만들고 다른 입력을 읽으며 해시 테이블을 탐색한다. 보통 작은 쪽을 Build Input으로 삼는 것이 메모리 부담을 줄인다.

정답 도출

고객 50만 건과 주문 2천만 건이라면 일반적으로 고객 쪽이 Build Input 후보가 된다. 물론 필터 조건, 조인 컬럼 분포, 필요한 컬럼, 병렬 분배 방식에 따라 실제 계획은 달라질 수 있다.

오답 분석

선택지	이유
①	SQL에 적은 순서가 Build Input을 결정하는 절대 기준은 아니다.
③	HASH JOIN은 대량 집합 조인에서 자주 유리하다.
④	Build Input이 메모리에 다 들어가지 않으면 Temp 사용과 성능 저하가 생길 수 있다.

정리

HASH JOIN에서는 Build Input 크기와 메모리 사용량을 반드시 확인한다.

문제 16 해설

정답: ②

핵심 개념

옵티마이저는 서브쿼리를 그대로 반복 실행하지 않고 조인 형태로 변환하거나 캐싱, 인덱스 탐색 등을 활용할 수 있다. 이를 쿼리 변환 관점에서 이해해야 한다.

정답 도출

문제 SQL은 고객별 최근 주문일자를 구한다. **주문(고객ID, 주문일자)** 인덱스가 있으면 고객별 MAX를 빠르게 찾을 수 있고, 경우에 따라 조인/집계로 변환하는 방식도 가능하다.

오답 분석

선택지	이유
①	항상 Full SCAN한다고 볼 수 없다. 인덱스, 캐싱, 변환에 따라 달라진다.
③	서브쿼리는 실행계획에서 세미 조인, 안티 조인, 일반 조인, 스칼라 서브쿼리 형태로 나타날 수 있다.
④	스칼라 서브쿼리는 한 로우 한 컬럼을 반환해야 하며 여러 건이면 오류가 발생할 수 있다.

정리

서브쿼리 성능은 문법 모양보다 옵티마이저가 어떤 실행 형태로 바꾸는지를 봐야 한다.

문제 17 해설

정답: ④

핵심 개념

UNION은 중복 제거를 포함한다. 중복 제거에는 정렬 또는 해시 작업이 필요할 수 있다. **UNION ALL**은 단순 결합이므로 중복 제거 비용이 없다.

정답 도출

업무 규칙상 두 집합에 중복이 없다면 **UNION**의 중복 제거는 불필요한 비용이다. 이런 경우 **UNION ALL**이 결과 의미를 유지하면서 더 가볍다.

오답 분석

선택지	이유
①	중복 없음이 보장되면 UNION ALL 을 적극 검토한다.
②	중복이 없으면 결과 건수는 같다. 중복이 있으면 UNION ALL 이 더 많다.
③	두 연산자는 중복 처리 방식과 실행 비용이 다르다.

정리

중복 제거가 필요하지 않은데 **UNION**, **DISTINCT**, 불필요한 **GROUP BY**를 쓰면 소트 비용이 생긴다.

문제 18 해설

정답: ③

핵심 개념

인덱스는 정렬된 구조다. 조건과 정렬 방향이 인덱스 구성과 맞으면 **ORDER BY** 소트를 생략하고 Top N에서 빨리 멈출 수 있다.

정답 도출

인덱스가 (종목코드, 체결일시 DESC, 체결가격)이고 SQL은 종목코드 = :code, **ORDER BY** 체결일시 DESC다. 특정 종목 범위 안에서 이미 최신순으로 정렬되어 있으므로 5건만 읽고 멈출 수 있다. 다만 **체결번호**는 인덱스에 포함되어 있지 않으므로, 최종 5건에 대해서만 **TABLE ACCESS BY INDEX ROWID**가 발생한다. 정렬 생략과 Stopkey 이점은 그대로 유지된다.

오답 분석

선택지	이유
①	Descending RANGE SCAN도 가능하다.
②	ORDER BY가 있어도 인덱스 순서를 활용하면 별도 Sort가 생략될 수 있다.
④	Stopkey 조건은 인덱스 순서와 결합될 때 큰 성능 이점을 준다.

정리

Top N은 정렬 생략 + 조기 중단이 핵심이다.

문제 19 해설

정답: ①

핵심 개념

Sort Area 부담은 정렬 대상 로우 수와 로우 폭에 영향을 받는다. 정렬 전에 불필요하게 큰 문자열을 만들면 메모리와 Temp 부담이 커질 수 있다.

정답 도출

A안은 정렬 전에 **line** 문자열을 크게 만든다. B안은 인라인 뷰에서 **상품번호, 상품명, 주문일시** 같은 원본 컬럼만 정렬한 뒤 바깥에서 문자열 포매팅을 수행한다. 정렬 대상의 행 폭을 줄일 수 있으므로 B안이 Sort Area 사용량을 줄이는 방향에 더 적절하다.

오답 분석

선택지	이유
②	문자열을 크게 만든 뒤 정렬하면 정렬 대상 폭이 커질 수 있다.
③	정렬 전에 만드는 데이터 폭이 다를 수 있다.
④	SQL 작성 방식은 정렬 입력량과 폭에 영향을 준다.

정리

정렬 전에는 행 수와 컬럼 폭을 줄이고, 포매팅은 가능하면 정렬 이후로 미룬다.

문제 20 해설

정답: ②

핵심 개념

DML 성능에는 인덱스와 Redo/Undo뿐 아니라 Database Call 횟수가 크게 영향을 준다. 한 건씩 호출하면 네트워크 왕복과 Parse/Execute 비용이 반복된다.

정답 도출

PreparedStatement를 루프 밖에서 만들고, 여러 건을 바인딩한 뒤 배치 실행하면 Database Call 횟수를 크게 줄일 수 있다. Array Processing은 대량 DML의 기본 개선 포인트다.

오답 분석

선택지	이유
①	한 건씩 처리한다고 Lock이 없어지지 않는다. 호출 비용만 늘 수 있다.
③	인덱스 추가는 INSERT 유지 비용을 늘릴 수 있고 Call 횟수와 무관하다.
④	Commit을 무작정 미루면 Undo, Lock 유지, 장애 복구 부담이 커질 수 있다.

정리

대량 DML은 “한 번에 많이 보내기”와 “불필요한 인덱스 유지 줄이기”를 함께 검토한다.

문제 21 해설

정답: ④

핵심 개념

Direct Path Insert는 버퍼 캐시를 경유하는 일반 경로와 달리 대량 적재에 유리할 수 있다. 보통 **APPEND**, 병렬 DML, 인덱스 후생성, 통계 수집과 함께 검토한다.

정답 도출

야간 대량 이관이고 동시 충돌이 없다면 Direct Path Insert가 적합한 후보가 된다. 다만 Redo/Undo, Lock, HWM 증가, 인덱스 관리, 통계정보 등 운영 조건을 함께 확인해야 한다.

오답 분석

선택지	이유
①	대량 적재 후 통계정보가 부정확하면 후속 SQL 실행계획이 나빠질 수 있다.
②	OLTP 단건 INSERT는 버퍼 캐시 재사용과 동시성이 중요하므로 일반적으로 Direct Path가 기본 선택은 아니다.
③	모든 부하가 0이 되는 것은 아니다. 모드와 옵션에 따라 Redo/Undo도 발생할 수 있다.

정리

Direct Path는 대량 처리 도구다. OLTP와 배치의 성격을 구분해야 한다.

문제 22 해설

정답: ③

핵심 개념

파티션은 관리 단위이자 성능 단위다. 대량 삭제를 로우 단위로 수행하지 않고 파티션 단위 작업으로 바꾸면 Undo/Redo, 인덱스 유지, 수행 시간을 크게 줄일 수 있다.

정답 도출

삭제 대상이 월 파티션과 정확히 대응하면 **TRUNCATE PARTITION** 또는 **DROP PARTITION**이 Row 단위 DELETE보다 훨씬 빠를 수 있다. 로컬 인덱스는 파티션과 1:1 대응하므로 관리 측면에서도 유리하다.

오답 분석

선택지	이유
①	로컬 인덱스는 파티션 구조 변경 시 관리 편의성이 높다. 글로벌 인덱스는 Unusable 이슈를 더 주의해야 한다.
②	파티션 단위 DDL로 대량 삭제를 대체할 수 있다.
④	파티션 Pruning과 파티션 단위 DML은 대량 처리 성능에 직접 도움이 된다.

정리

시계열 대량 데이터는 “인덱스로 빨리 지우기”보다 “파티션 단위로 제거하기”가 더 강력할 수 있다.

문제 23 해설

정답: ①

핵심 개념

채번은 동시성 제어와 직결된다. **max + 1** 방식은 여러 세션이 같은 최댓값을 읽고 같은 다음 번호를 계산할 수 있어 충돌과 직렬화가 필요하다.

정답 도출

동시 주문이 많은 시스템에서는 같은 일자의 최댓값을 읽는 세션들이 몰린다. 중복을 막으려면 별도 Lock이나 직렬화가 필요하고, 이 지점이 INSERT 병목이 된다. 시퀀스나 별도 채번 설계를 검토한다.

오답 분석

선택지	이유
②	max + 1 은 직렬화 병목이 생기기 쉬워 고동시성에 취약하다.
③	인덱스는 max 탐색을 빠르게 할 수 있지만 동시성 충돌 자체를 제거하지 않는다.
④	채번 방식은 INSERT 처리량과 Lock 경합에 큰 영향을 준다.

정리

성능 좋은 채번은 빠른 조회보다 경합 없는 동시 처리가 핵심이다.

문제 24 해설

정답: ④

핵심 개념

선택도는 전체 로우 중 조건을 만족할 것으로 예상되는 비율이고, 카디널리티는 예상 로우 수다. 균일 분포의 등치 조건은 단순히 **1 / NDV**로 추정할 수 있다.

정답 도출

선택도 = $1 / 100 = 0.01$
카디널리티 = $10,000,000 \times 0.01 = 100,000$

따라서 약 10만 건이다.

오답 분석

선택지	이유
①	NDV가 1,000일 때의 단순 추정에 가깝다.
②	조건이 전혀 선택력을 갖지 않을 때에 가깝다.
③	NDV가 10일 때의 단순 추정에 가깝다.

정리

옵티마이저의 비용 계산은 선택도와 카디널리티 추정에서 출발한다. 데이터 분포가 균일하지 않으면 히스토그램 등 추가 통계가 중요해진다.

문제 25 해설

정답: ②

핵심 개념

예상 실행계획은 옵티마이저의 추정이다. 실제 수행 중 각 단계가 몇 번 실행되었고 얼마나 읽었는지는 Row Source 통계를 봐야 한다.

정답 도출

gather_plan_statistics 힌트 또는 세션 통계 설정 후 다음처럼 확인할 수 있다.

```
SELECT *  
FROM TABLE(dbms_xplan.display_cursor(NULL, NULL, 'ALLSTATS LAST'));
```

이 방식은 **Starts, E-Rows, A-Rows, Buffers, Reads** 등을 통해 예상과 실제의 차이를 볼 수 있다.

오답 분석

선택지	이유
①	EXPLAIN PLAN은 실제 실행 통계를 보여주지 않는다.
③	V\$SQL은 누적 통계로 튜닝 대상 선정에는 좋지만 실행계획 단계별 병목을 직접 보여주지는 않는다.
④	실제 성능 문제는 예상 계획만으로 설명되지 않는 경우가 많다.

정리

분석 도구는 목적별로 고른다. 예상 계획은 빠른 확인, SQL Trace와 DBMS_XPLAN ALLSTATS는 실제 병목 분석, V\$SQL은 누적 통계 기반 대상 선정에 강하다.